



COMILLAS
UNIVERSIDAD PONTIFICIA

ICAI

ICADE

CIHS

Procesamiento Streaming

Tecnología de Datos Masivos /
Big Data Technology

comillas.edu

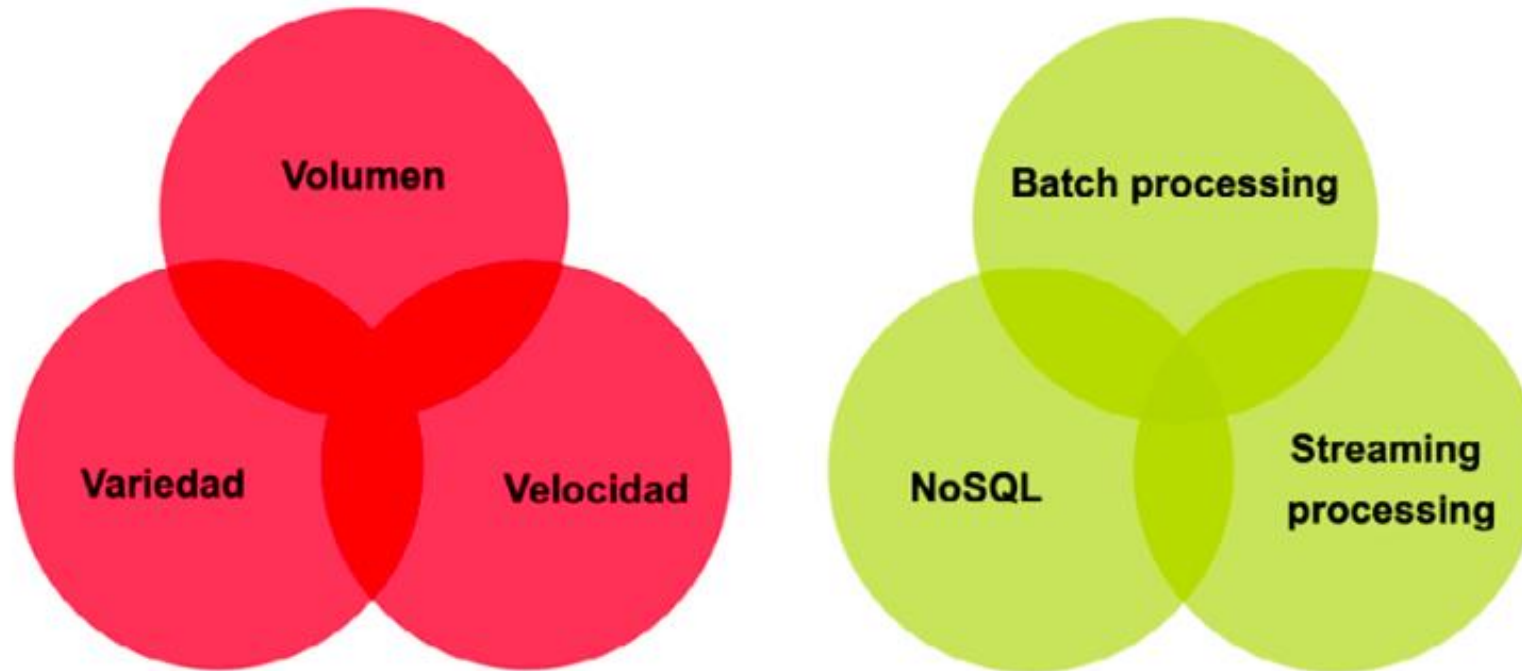
01	INTRODUCCIÓN
02	APACHE KAFKA
03	SPARK STREAMING
04	ELASTICSEARCH
05	KIBANA
06	DEMO

01	INTRODUCCIÓN
02	APACHE KAFKA
03	SPARK STREAMING
04	ELASTICSEARCH
05	KIBANA
06	DEMO

01 | INTRODUCCIÓN

Introducción

Con el objetivo de dar solución a los problemas relacionados con el volumen, la velocidad y la variedad, surgió la necesidad de desarrollar nuevas arquitecturas y herramientas.



01 | INTRODUCCIÓN

Introducción

Para los problemas de **volumen y velocidad**, tenemos, en la actualidad, un amplio conjunto de **herramientas de computación distribuida**.

El otro problema, la **variedad**, se resuelve con las tecnologías de almacenamiento de información **NoSQL**.

Estas herramientas de computación distribuida se pueden clasificar de acuerdo con tres **paradigmas de procesamiento**.

Batch

Streaming

Híbrido

Un factor que diferencia los procesamientos batch y streaming es la **latencia**. La latencia es el tiempo que transcurre entre que hacemos una consulta de información y obtenemos la respuesta.

01 | INTRODUCCIÓN

Procesamiento Batch

El procesamiento batch o por lotes da solución al problema del volumen.

Características	Herramientas
• Alta latencia • Escalable • Manejo de grandes cantidades de información estática • Distribuido • Ejecución en paralelo • Tolerante a fallos	• Hadoop MapReduce • Apache Spark • Apache Hive • Apache Pig

01 | INTRODUCCIÓN

Procesamiento Streaming

El procesamiento streaming da solución al problema de la velocidad.

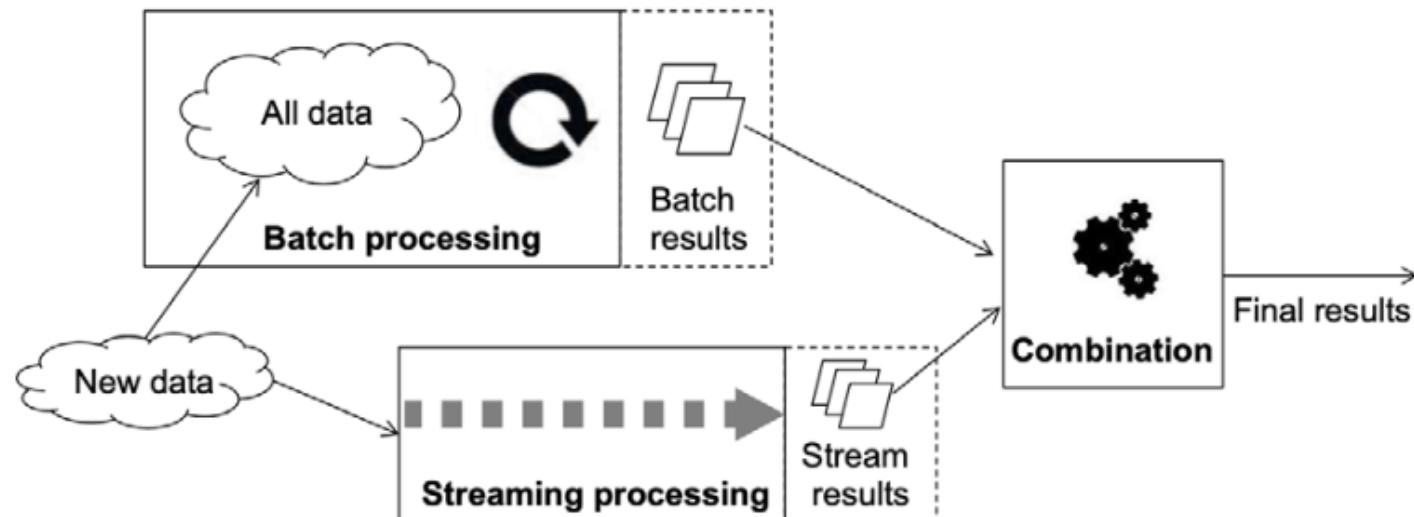
Características	Herramientas
• Baja latencia • Información generada de continuo (Streams). • Distribuido • Ejecución en paralelo • Tolerante a fallos	• Apache Spark Streaming • Kafka Streams • Apache Storm

01 | INTRODUCCIÓN

Procesamiento Híbrido

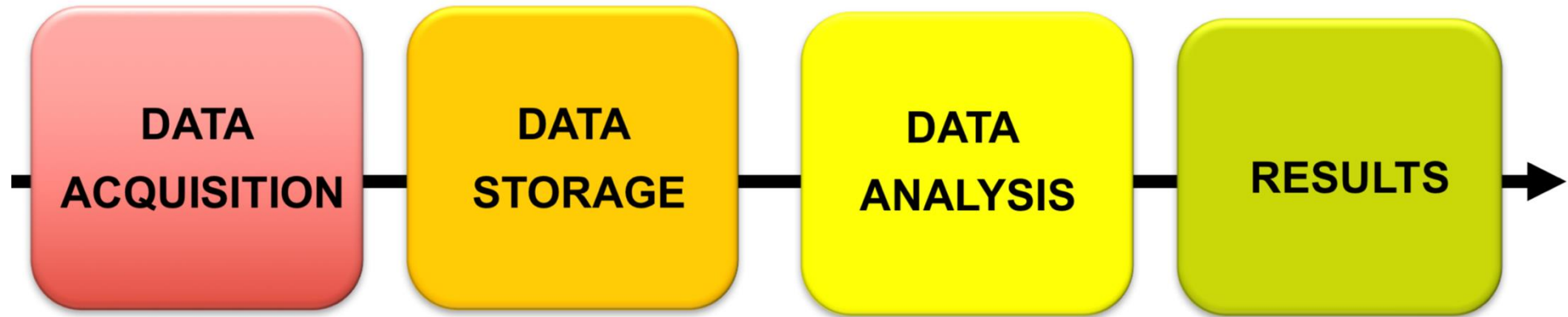
El procesamiento híbrido surgió de la necesidad de dar solución a ambos problemas a la vez, volumen y velocidad.

Características	Herramientas
• Baja latencia • Datos en estático + Datos en movimiento • Escalable • Combina resultados batch y streaming.	• Apache Flink



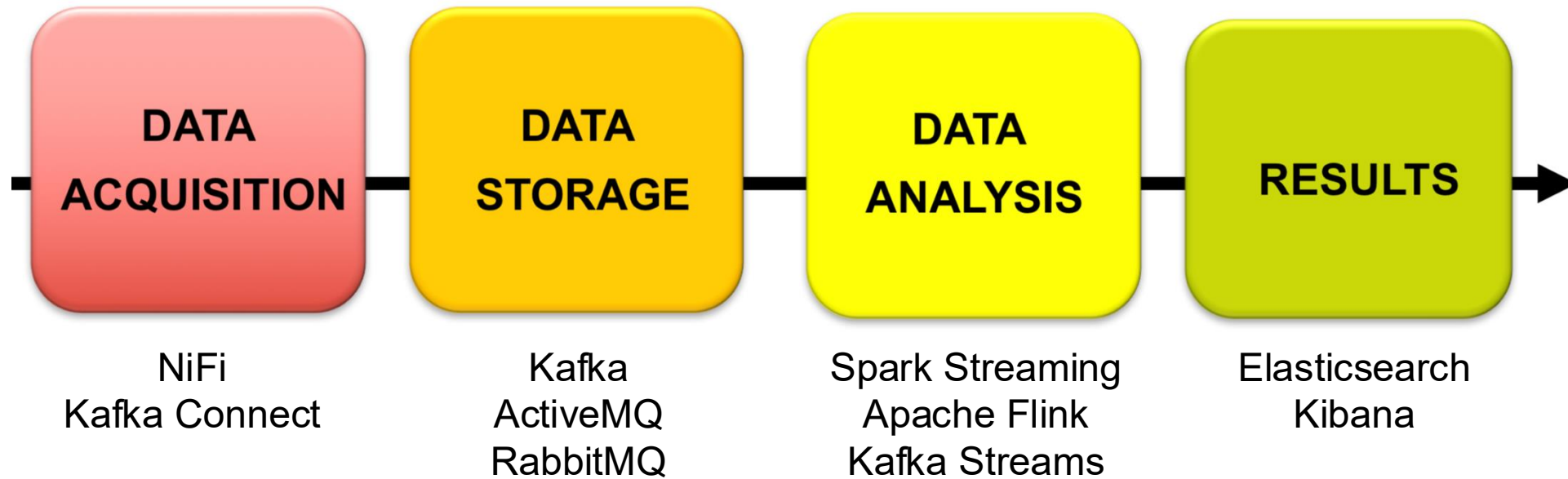
01 | INTRODUCCIÓN

Flujo de Procesamiento



01 | INTRODUCCIÓN

Tecnologías Procesamiento Streaming



01	INTRODUCCIÓN
02	APACHE KAFKA
03	SPARK STREAMING
04	ELASTICSEARCH
05	KIBANA
06	DEMO

02 | APACHE KAFKA

Introducción

Apache Kafka es una plataforma de streaming de eventos distribuida, diseñada para manejar flujos de datos en tiempo real. Fue desarrollada originalmente por LinkedIn y se ha convertido en una herramienta esencial para las arquitecturas Big Data. Kafka permite publicar, suscribirse, almacenar y procesar eventos de manera eficiente y escalable.

- **Características Clave**

- Escalabilidad Horizontal y Vertical: Kafka puede manejar grandes volúmenes de datos sin sacrificar el rendimiento.
- Durabilidad y Tolerancia a Fallos: Los datos se replican en varios nodos del clúster, asegurando que no se pierdan incluso si un nodo falla.
- Latencia Baja: Permite acceder a datos en tiempo real, facilitando la toma de decisiones rápidas.
- Flexibilidad: Se adapta a diversas necesidades, desde pequeñas implementaciones hasta grandes flujos de datos en escala empresarial.

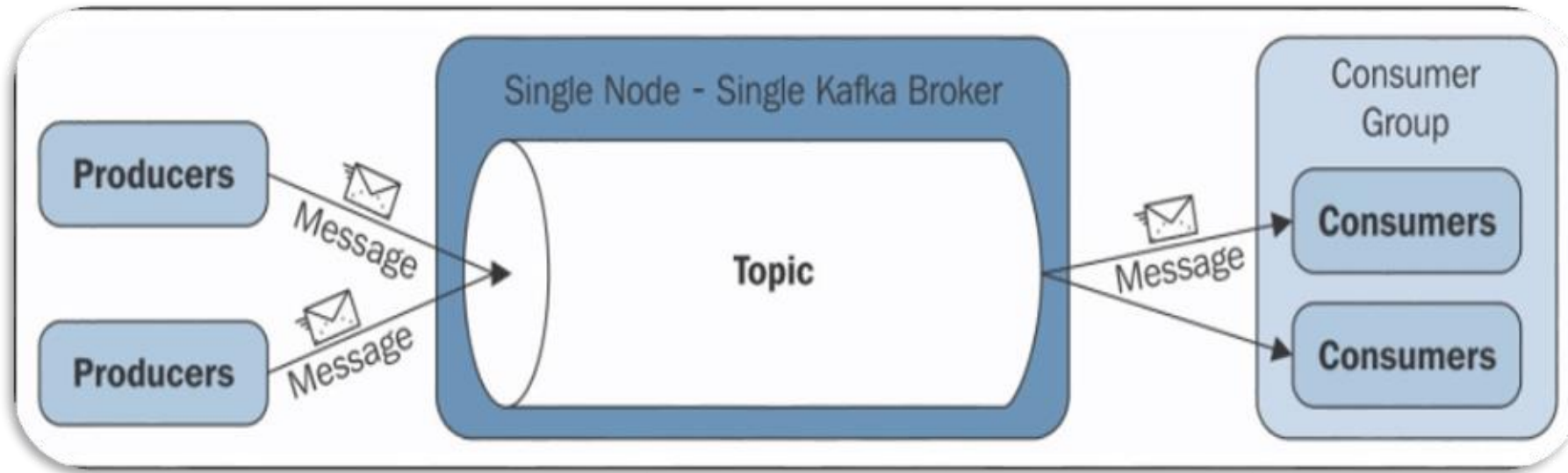


Saber más:
[Apache Kafka](#)
[Confluent](#)

02 | APACHE KAFKA

Componentes principales

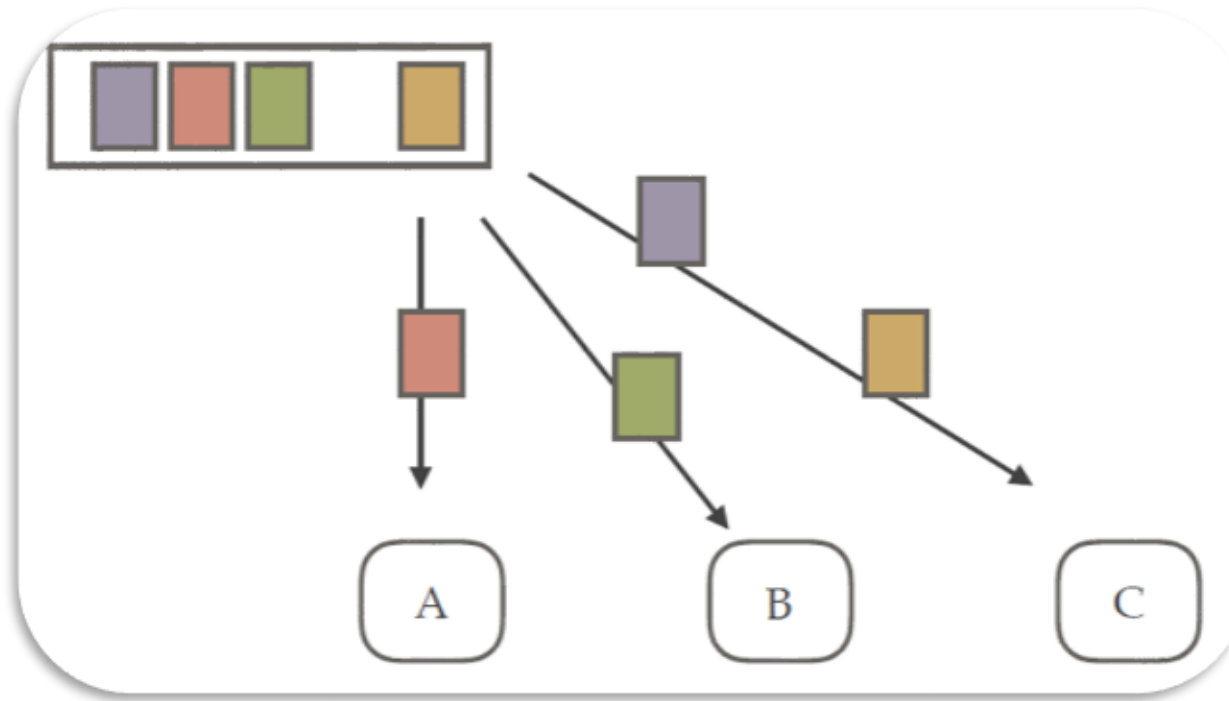
- **Topics:** Canales de comunicación donde se categorizan los mensajes. Cada topic está dividido en particiones
- **Producers:** Generan y envían mensajes a los topics.
- **Consumers:** Se suscriben a los topics y procesan los mensajes según sus necesidades.
- **Brokers:** Servidores que almacenan los mensajes y gestionan las particiones.



02 | APACHE KAFKA

Modelo de colas

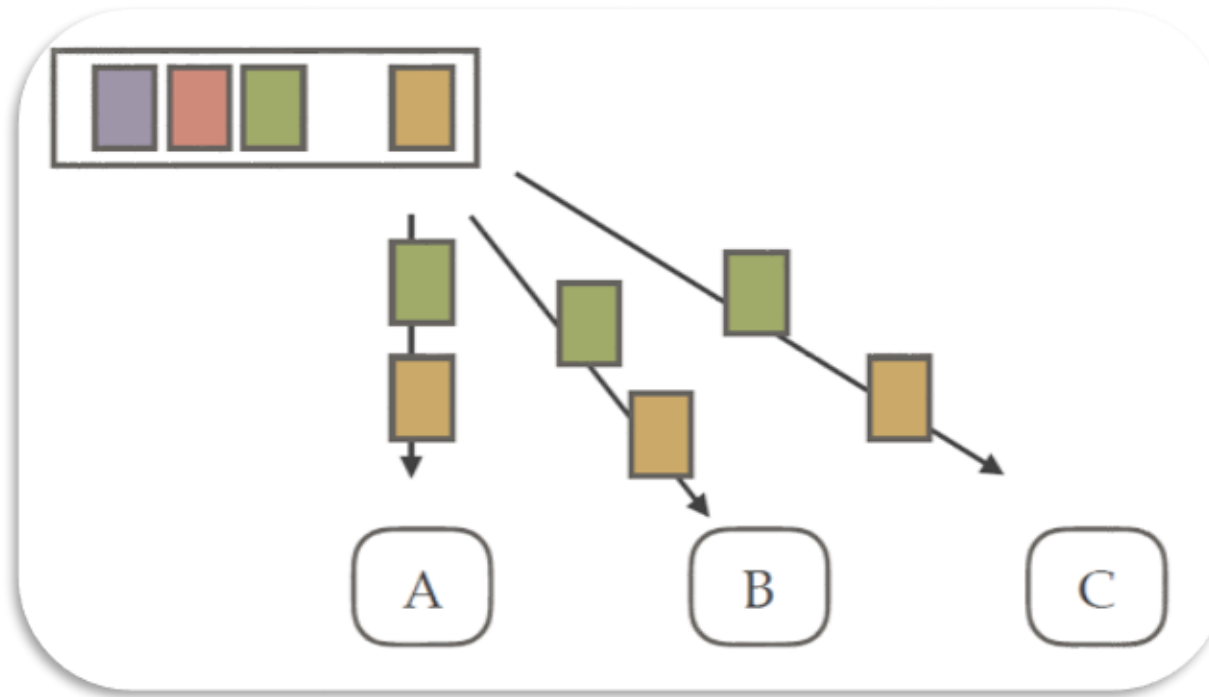
- Los mensajes son repartidos y cada mensaje es leído por un consumer.



02 | APACHE KAFKA

Modelo de publicador/subscriptor

- Los mensajes se difunden en **broadcast** y todos los mensajes son leídos por todos los consumers.



02 | APACHE KAFKA

Brokers

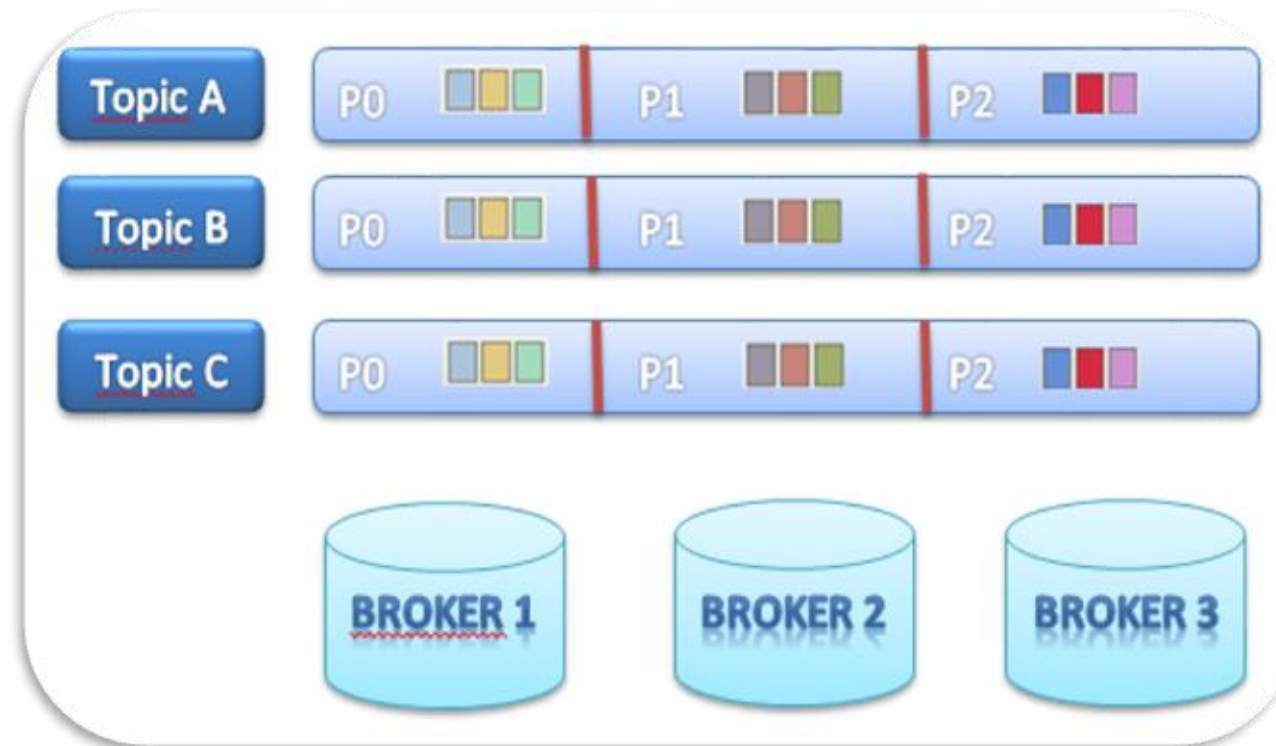
- Es el servicio principal de Kafka y cada nodo de Kafka tendrá un broker.
- Almacenan las distintos topics (o colas de mensajes), gestionan las particiones de los topics, escriben los datos en disco y se encargan de la securización de los topics.
- Se utilizan para crear clúster de Kafka.
- La sincronización se consigue gracias a la utilización de Zookeeper.



02 | APACHE KAFKA

Particiones

- Los topics se dividen en **particiones** donde se almacenan los mensajes.
- Cada partición es un secuencia **ordenada** e inmutable de mensajes y está asociada a un broker.
- Al leer/escribir mensajes de los topics, las operaciones se realizan a nivel de partición y únicamente sobre la partición líder.



02 | APACHE KAFKA

Particiones

- Cada partición es una carpeta (llamada **logs**) en disco. Las particiones se dividen en **segmentos** y cada segmento es un fichero en disco (llamado **log**).
- Internamente cada mensaje es identificado por un **offset**.
- Las particiones se distribuyen entre los brokers del clúster de Kafka y se utilizan para balancear los mensajes a través de los brokers.

```
/tmp/kafka-logs/topic1-0:
```

```
-rw-r--r-- 1 root root 10485760 dic 7 17:01 00000000000000000000.index
```

```
-rw-r--r-- 1 root root          0 dic 7 17:01 00000000000000000000.log
```

```
/tmp/kafka-logs/topic1-1:
```

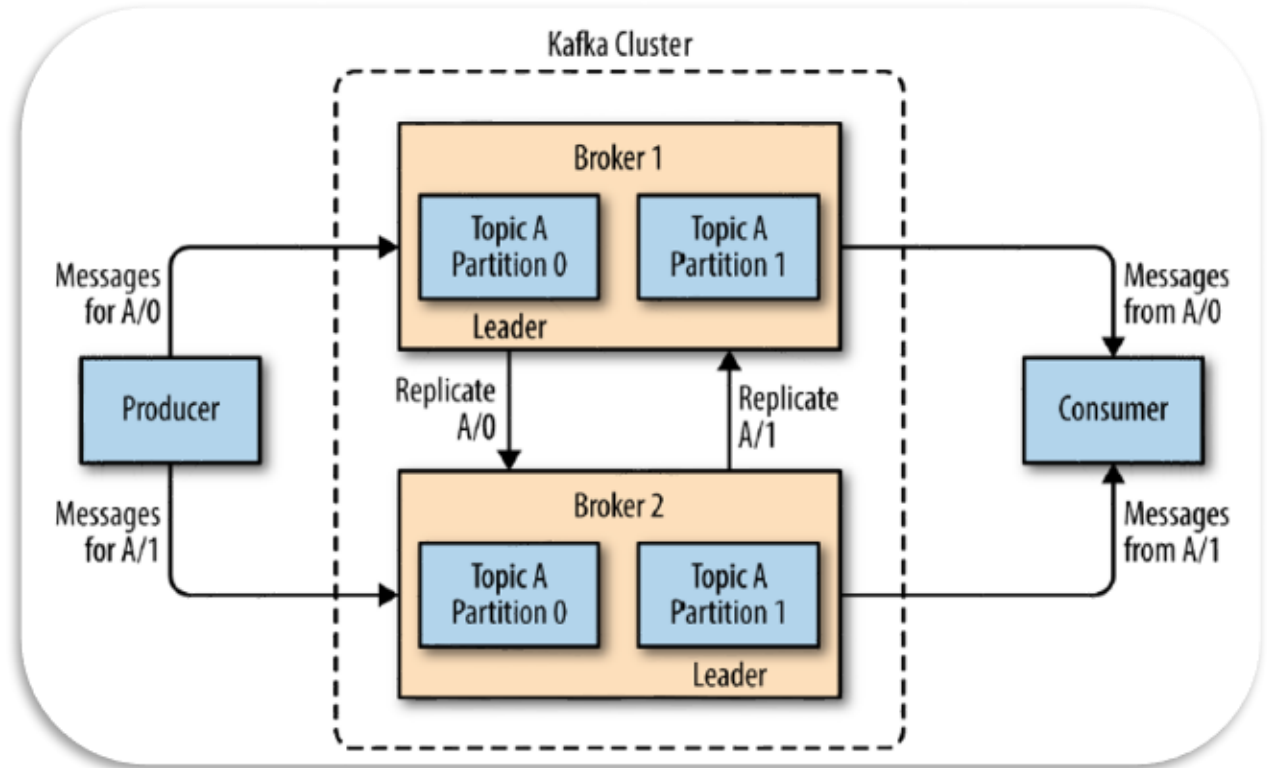
```
-rw-r--r-- 1 root root 10485760 dic 7 17:01 00000000000000000000.index
```

```
-rw-r--r-- 1 root root      175 dic 7 17:10 00000000000000000000.log
```

02 | APACHE KAFKA

Réplicas

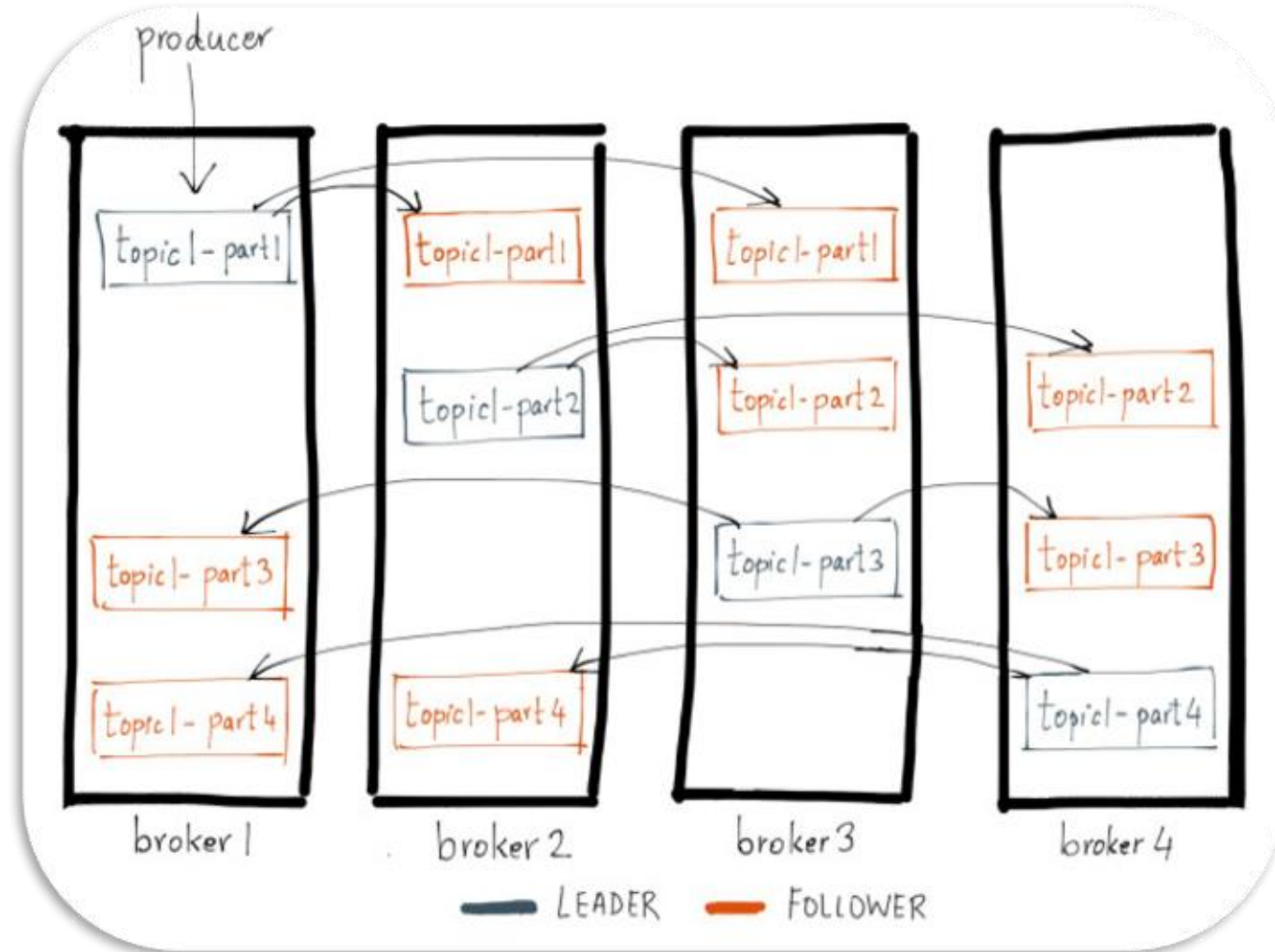
- Los topics de Kafka tienen la posibilidad de configurar **réplicas**.
- Las réplicas se utilizan para conseguir alta disponibilidad del servicio.
- Los datos se replican a nivel de partición, duplicando las particiones entre los diferentes brokers del clúster.
- Podrá haber tantas réplicas como número de brokers.
- Al incrementar las réplicas hacemos el sistema más robusto ante caídas de los brokers.
- Las réplicas implican un incremento del ancho de banda entre los brokers.
- Dependiendo de la configuración del ACK el throughput de mensajes podrá verse disminuido



02 | APACHE KAFKA

Réplicas

- Configuración de un topic con 4 particiones y réplica 3.



- **Leader réplica**
 - Para mantener la consistencia de los datos, todas las peticiones de lectura/escritura se realizan en la réplica leader.
 - También se encarga de saber qué replicas follower están in-sync.
 - La primera réplica viva se convierte en la réplica leader.
- **Réplicas Follower**
 - No gestionan operaciones de lectura/escritura.
 - Su única función es replicar los mensajes desde la réplica leader para mantenerse sincronizada con los mensajes.

02 | APACHE KAFKA

Mensajes

- Los **mensajes** están formados por:
 - Timestamp
 - Clave
 - Valor
- La clave suele ser usada para decidir a qué partición se envía cada mensaje.
- Un mensaje se localiza por el **offset X de la partición Y del topic Z**

02 | APACHE KAFKA

Producers

- Son los encargados de enviar los mensajes a los topics.
- Los mensajes se envían directamente al broker de la réplica leader.
- Los mensajes se pueden enviar utilizando un partitioner, que decide la partición a la que se enviará el mensaje.
- El envío de mensajes se realiza en batchs de mensajes basados en tiempo o en el número de mensajes.
- Se puede utilizar envío síncrono o asíncrono.

02 | APACHE KAFKA

Consumers



- Son los encargados de leer los mensajes de los topics.
- Se pueden agrupar en **consumers groups**.
- Utilizan un topic especial **__consumer_offsets** para guardar el **offset** del último mensaje leído de cada partición. En caso de fallo, un consumer seguiría leyendo desde el último offset guardado.
- Dentro de una partición, cada mensaje tiene un offset único.

01	INTRODUCCIÓN
02	APACHE KAFKA
03	SPARK STREAMING
04	ELASTICSEARCH
05	KIBANA
06	DEMO

03 | SPARK STREAMING

Introducción

Spark Streaming es un módulo de Apache Spark que permite el procesamiento de flujos de datos en tiempo real. Transforma flujos de datos en mini-batches y los procesa utilizando las mismas operaciones que en Spark.

- **Características Clave**

- Micro-batching: Los datos en tiempo real se dividen en pequeños lotes (micro-batches) que se procesan en intervalos regulares.
- Integración con Spark Core: Utiliza las mismas API y operaciones que Spark Core, facilitando la transición entre procesamiento en batch y en tiempo real.
- Tolerancia a Fallos: Recuperación automática de fallos mediante la replicación de datos y el uso de logs de transacciones.

Saber más:

[Spark Streaming Programming Guide](#)

- **Ventajas de Usar Spark Streaming**

- Unificación: Permite unificar el procesamiento de datos en batch y en tiempo real en una sola plataforma.
- Escalabilidad: Capacidad para escalar horizontalmente y manejar grandes volúmenes de datos en tiempo real.
- Facilidad de Uso: API sencilla y consistente con el resto de Spark.
- Integración: Compatible con diversas fuentes de datos y sistemas de almacenamiento.

- **Casos de Uso:**

- Monitorización de Logs: Análisis en tiempo real de logs de servidores para detectar anomalías.
- Análisis de Clickstream: Procesamiento de datos de clics en tiempo real para análisis de comportamiento de usuarios.
- Detección de Fraude: Identificación de patrones de fraude en transacciones financieras en tiempo real.
- Análisis de Redes Sociales: Procesamiento de flujos de datos de redes sociales para análisis de tendencias y sentimientos.

Spark Structured Streaming es el motor de procesamiento de datos en streaming de Apache Spark basado en **Spark SQL y la API de DataFrames/Datasets**. Permite procesar flujos de datos como si fueran **tablas infinitas** sobre las que se ejecutan consultas declarativas, que se actualizan continuamente a medida que llegan nuevos datos. El desarrollador escribe consultas similares a las de procesamiento batch, mientras que Spark se encarga de ejecutarlas **incrementalmente y de forma continua** sobre los datos en streaming.

• Características Clave

- Modelo declarativo basado en DataFrames y SQL
- Procesamiento incremental automático
- Garantías de procesamiento exactly-once
- Gestión automática de estado y tolerancia a fallos
- Procesamiento en micro-batches o modo continuo
- Integración con el ecosistema Spark (MLlib, SQL, Delta, etc.)

• Ventajas de Usar Spark Streaming

- Modelo de programación unificado para batch y streaming
- Menor complejidad para el desarrollador (no necesita gestionar manualmente el estado del stream)
- Escalabilidad distribuida aprovechando el motor de Spark
- Alta fiabilidad mediante checkpointing y logs de recuperación
- Integración sencilla con múltiples fuentes y sinks (Kafka, archivos, bases de datos, data lakes, etc.)

• Casos de Uso:

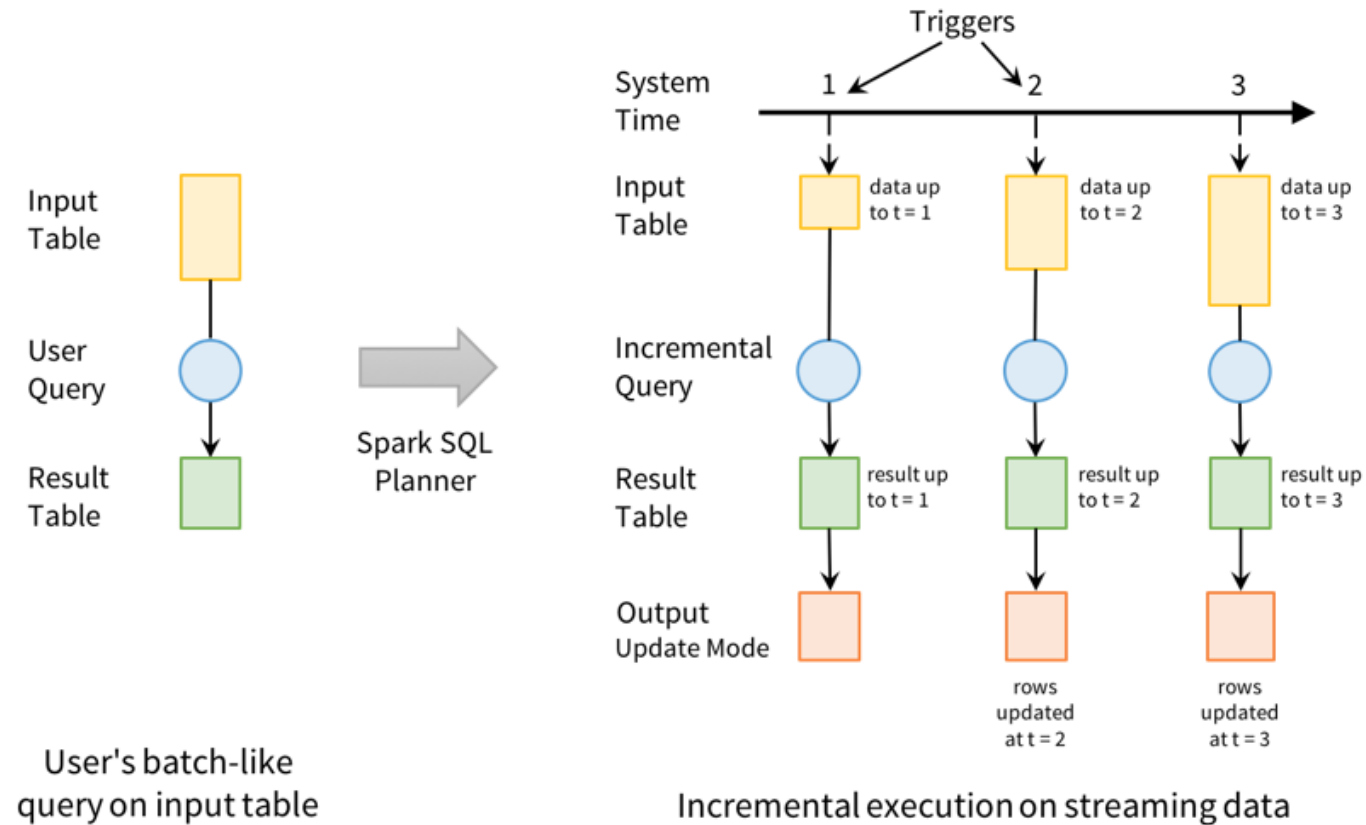
- Procesamiento de eventos en tiempo real (logs, clickstreams)
- Streaming ETL hacia data lakes
- Monitorización de IoT y sensores
- Análisis financiero o de trading en tiempo real
- Detección de fraude o anomalías

Saber más:

[Structured Streaming Programming Guide](#)

03 | SPARK STREAMING

Structured Streaming



Structured Streaming Processing Model

Users express queries using a batch API; Spark incrementalizes them to run on streams

01	INTRODUCCIÓN
02	APACHE KAFKA
03	SPARK STREAMING
04	ELASTICSEARCH
05	KIBANA
06	DEMO

Elasticsearch es un **motor de búsqueda y análisis distribuido**, de código abierto, basado en **Apache Lucene**. Es ampliamente utilizado para búsquedas de texto completo, almacenamiento y análisis de grandes volúmenes de datos en tiempo real

- **Características Clave**

- **Búsqueda rápida y eficiente:** Diseñado para proporcionar capacidades de búsqueda ultrarrápidas en grandes volúmenes de datos
- **Escalabilidad distribuida:** Permite el almacenamiento y procesamiento de datos en múltiples nodos, mejorando la tolerancia a fallos y el rendimiento
- **API RESTful:** Facilita la integración con diversas aplicaciones y lenguajes de programación mediante solicitudes HTTP
- **Análisis de texto natural:** Ofrece funciones avanzadas para procesar y buscar texto en lenguaje natural
- **Compatibilidad con diversos tipos de datos:** Admite datos textuales, numéricos, geoespaciales, estructurados y no estructurados
- **Integración con Kibana y Logstash:** Parte del Elastic Stack, junto con Kibana para visualización y Logstash para procesamiento de datos



Saber más:
[Elastic Stack](#)
[Elasticsearch](#)

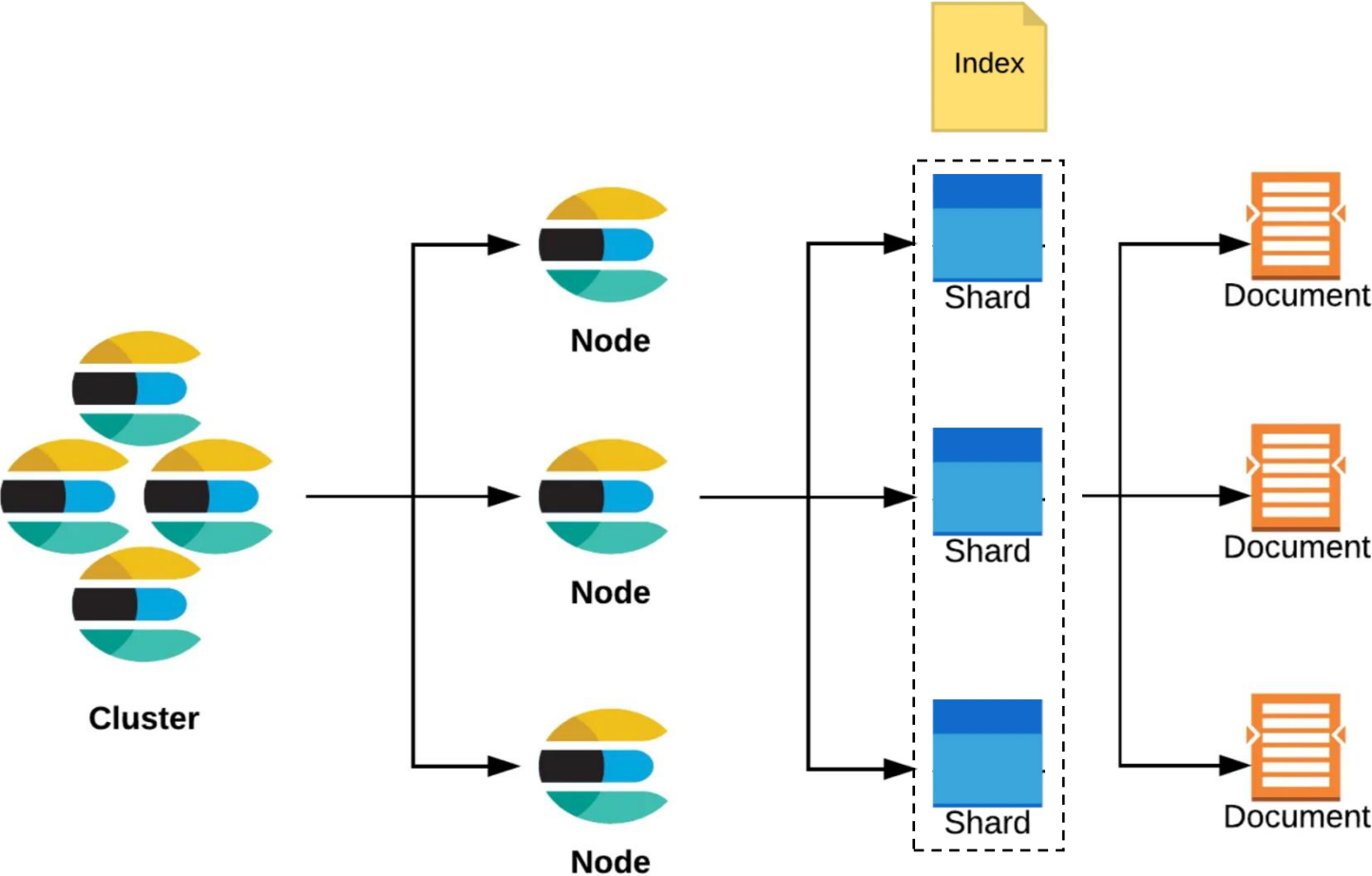
04 | ELASTICSEARCH

Componentes principales

- **Nodos y Clústeres**
 - Un nodo es un servidor que almacena datos y participa en el clúster.
 - Un clúster es un conjunto de nodos que trabajan juntos
- **Índices**
 - Unidad lógica para organizar y almacenar documentos.
 - Cada índice se compone de uno o más Shards
- **Shards**
 - Los datos se dividen en fragmentos llamados Shards.
 - Los fragmentos permiten una indexación, consulta y distribución de datos más rápida.
 - Cada fragmento contiene un subconjunto de los documentos almacenados en el índice. Un índice puede dividirse en uno o más fragmentos. Se distribuyen a diferentes nodos dentro del clúster, procesando la distribución de datos en paralelo para mejorar el rendimiento.
 - Son replicados para tener **alta disponibilidad**
- **Documentos**
 - Representa la unidad fundamental de datos. El concepto de "documento" puede considerarse el equivalente a una "fila" en la estructura de una base de datos relacional. Los campos de un documento pueden considerarse el equivalente a las "columnas" de una base de datos relacional.
 - Un documento es una colección de datos estructurados en formato **JSON**. Elasticsearch utiliza estos documentos para el almacenamiento y la consulta de datos. Cada documento tiene un **ID único**.

04 | ELASTICSEARCH

Arquitectura



01	INTRODUCCIÓN
02	APACHE KAFKA
03	SPARK STREAMING
04	ELASTICSEARCH
05	KIBANA
06	DEMO

Kibana es una plataforma de visualización de datos de código abierto desarrollada por Elastic. Permite a los usuarios explorar, visualizar y analizar datos almacenados en Elasticsearch. Es una herramienta esencial dentro del Elastic Stack, que también incluye Elasticsearch y Logstash

- **Características Clave**

- **Visualización de Datos:** Ofrece una amplia variedad de opciones de visualización, como gráficos de barras, gráficos circulares, mapas geográficos y tablas.
- **Exploración de Datos:** Permite realizar búsquedas y aplicar filtros avanzados para explorar datos de manera eficiente.
- **Dashboards Personalizados:** Los usuarios pueden crear y compartir dashboards interactivos que muestran información en tiempo real.
- **Alertas y Notificaciones:** Configuración de alertas para recibir notificaciones sobre eventos específicos en los datos.
- **Seguridad y Autenticación:** Funciones avanzadas de seguridad para proteger los datos y controlar el acceso



Saber más:
[Kibana](#)

05 | KIBANA

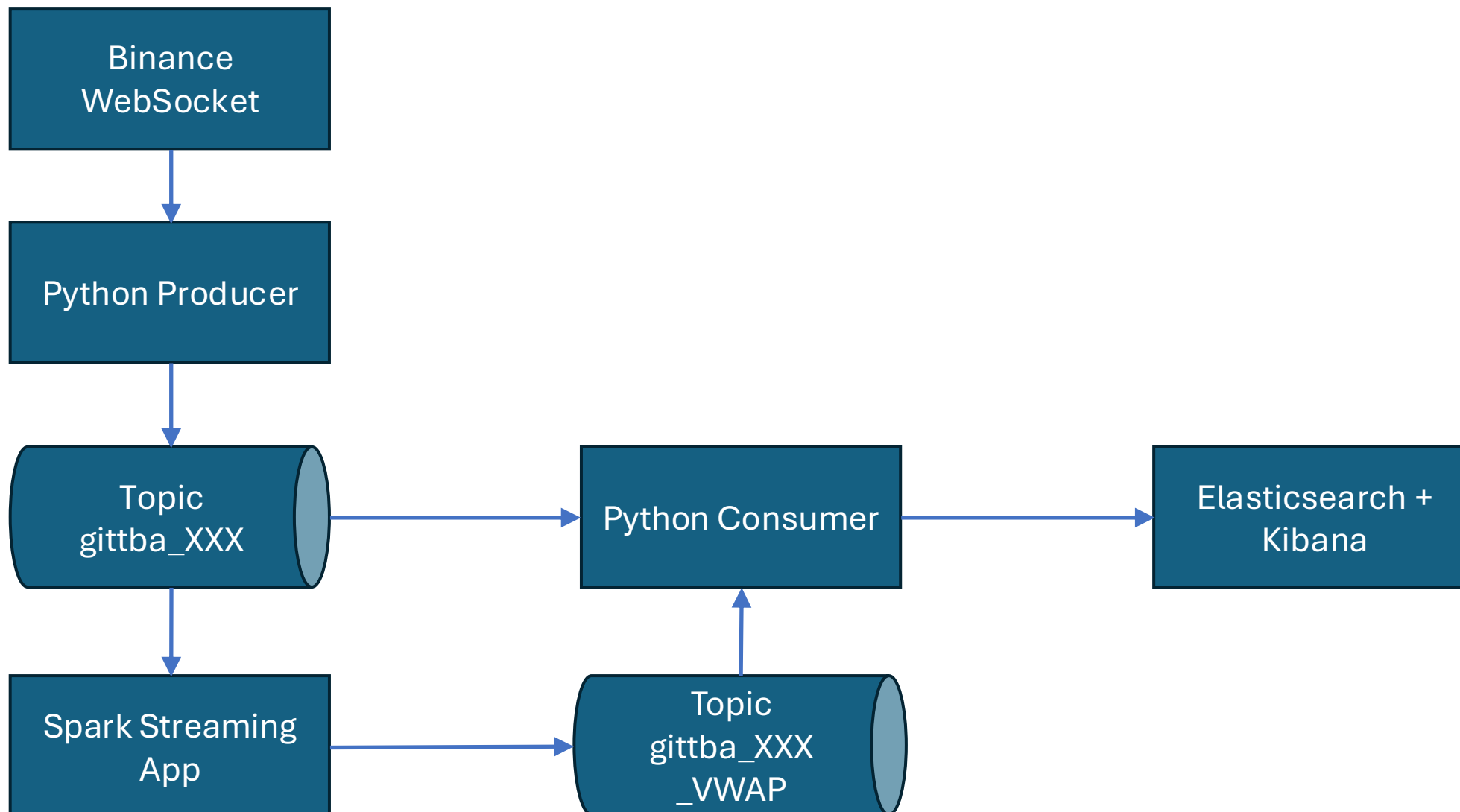
Componentes principales

- **Discover**
 - Herramienta para explorar datos de Elasticsearch de manera interactiva.
- **Visualize**
 - Permite crear una amplia gama de visualizaciones gráficas a partir de los datos.
- **Dashboards**
 - Creación y gestión de dashboards que contienen diversas visualizaciones.
- **Dev Tools**
 - Interfaz para interactuar directamente con la API de Elasticsearch

01	INTRODUCCIÓN
02	APACHE KAFKA
03	SPARK STREAMING
04	ELASTICSEARCH
05	KIBANA
06	DEMO

06 | DEMO

Arquitectura propuesta



06 | DEMO

Apache Kafka

- Conectar al modo Edge01 del clúster
 - `ssh gittba_bdtxx@192.168.80.33`
- Crear un topic (Cada grupo crea el suyo, sustituyendo **xx** por el número del grupo)
 - `kafka-topics --create --zookeeper 192.168.80.32:2181 --replication-factor 1 --partitions 1 --topic gittbaxx_test`
- Listar los topics
 - `kafka-topics --zookeeper 192.168.80.32:2181 --list | grep gittba`
- Describir un topic
 - `kafka-topics --zookeeper 192.168.80.32:2181 --describe --topic gittbaxx_test`
- Borrar un topic
 - `kafka-topics --delete --zookeeper 192.168.80.32:2181 --topic gittbaxx_test`

06 | DEMO

Apache Kafka

- Crear un producer (consola)
 - `kafka-console-producer --broker-list 192.168.80.34:9092 --topic gittbaxx_test --property parse.key=true --property key.separator=,`
- Crear un consumer (consola)
 - `kafka-console-consumer --bootstrap-server 192.168.80.34:9092 --topic gittbaxx_test --property print.key=true --consumer-property group.id=gittbaxx_group1`
- Listar los Consumer Groups
 - `kafka-consumer-groups --bootstrap-server 192.168.80.34:9092 --list`
- Ver la posición de un consumer group en cada topic/partition
 - `kafka-consumer-groups --bootstrap-server 192.168.80.34:9092 --describe --group gittbaxx_group1`

06 | DEMO

Apache Kafka

- Crear y ejecutar un producer y un consumer con python
 - `pip3 install kafka-python`
 - `python3 kafka_simple_producer.py`
 - `python3 kafka_simple_consumer.py`

06 | DEMO

Structured Streaming

- # 1. Crear la sesión de Spark. Utilizar las librerías correspondientes a la **versión de Spark** utilizada

```
spark = SparkSession.builder \  
    .appName("CryptoVWAPCalculator") \  
    .config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-10_2.13:4.1.1") \  
    .getOrCreate()
```

- # 2. Leer datos del topic de Kafka de entrada

```
df_kafka_input = spark.readStream \  
    .format("kafka") \  
    .option("kafka.bootstrap.servers", BOOTSTRAP_SERVERS) \  
    .option("subscribe", TOPIC_INPUT) \  
    .load()
```

06 | DEMO

Elasticsearch

- Conectar al modo Edge01 del clúster (Cada alumno con su usuario)
`ssh gittba_bdtxx@192.168.80.33`
- Comprobar el estado del clúster
`curl -X GET http://192.168.80.37:9201/_cat/health?v -u user:password`
- Listar los índices creados
`curl -X GET http://192.168.80.37:9201/_cat/indices?v -u user:password`
- Crear un índice (Cada grupo crea el suyo, sustituyendo **xx** por el número del grupo)
`curl -X PUT http://192.168.80.37:9201/gittbaxx_test?pretty -u user:password`
- Obtener información de un índice
`curl -X GET http://192.168.80.37:9201/gittbaxx_test?pretty -u user:password`

06 | DEMO

Elasticsearch

- Añadir un documento al índice creado. Cada documento debe tener un identificador único (Ej: **1**)

```
curl -X PUT http://192.168.80.37:9201/gittbaxx_test/_doc/1?pretty \  
-H 'Content-Type: application/json' \  
-d '{"symbol": "BTCUSDT", "@timestamp": "2026-03-09T11:21:00Z", "close": 67396.99, "volume": 19.5 }' \  
-u user:password
```
- Consultar un documento

```
curl -X GET http://192.168.80.37:9201/gittbaxx_test/_doc/1?pretty -u user:password
```
- Realizar una query de búsqueda

```
curl -X GET http://192.168.80.37:9201/gittbaxx_test/_search/?pretty \  
-H 'Content-Type: application/json' \  
-d '{"query": {"match": {"symbol": "BTCUSDT"} } }' \  
-u user:password
```
- Borrar un documento

```
curl -X DELETE http://192.168.80.37:9201/gittbaxx_test/_doc/1 -u user:password
```
- Borrar un índice

```
curl -X DELETE http://192.168.80.37:9201/gittbaxx_test -u user:password
```

06 | DEMO

Elasticsearch

- Elasticsearch con python
 - python3 elastic_get.py
 - python3 elastic_put.py

06 | DEMO

Kibana

- Crear **DataViews** y **Dashboards** en **Kibana** para visualizar los resultados almacenados en un índice de **Elasticsearch**
- Acceder a Kibana
 - Abrir Kibana en el navegador. <http://192.168.80.37:5601>
- Ver índices creados
 - En el menú lateral seleccionar “Management” y luego “Stack Management”
 - En el menú lateral seleccionar “Data” y luego “Index Management”
- Crear un Data View
 - En el menú lateral seleccionar “Management” y luego “Stack Management”
 - En el menú lateral seleccionar “Kibana” y luego “Data Views”
 - Hacer clic en “Create data view”.
 - Introducir el nombre del índice y el patrón que coincida con el índice, como gittbaxx_test*).
 - Seleccionar el campo @timestamp como el campo de tiempo.

Create data view

Name

gittba00_test

Index pattern

gittba00_test*

Timestamp field

@timestamp

Select a timestamp field for use with the global time filter.

[Show advanced settings](#)

06 | DEMO

Kibana

- Explorar los datos
 - En el menú lateral seleccionar “Analytics” y luego “Discover”
 - Seleccionar el Data View recién creado.
 - Ver y explorar los documentos.
- Crear el Dashboard
 - En el menú lateral seleccionar “Analytics” y luego “Dashboard”
 - Hacer clic en “Create dashboard”
 - Hacer clic en “Create visualization”
 - Seleccionar el Data View y configurar los parámetros de la visualización (ejes, métricas, etc.).
 - Elegir el tipo de visualización (gráfico de barras, líneas, etc.).
- Guardar
 - Guardar el dashboard para futuras referencias.